

STATISTICAL LEARNING

Chapter 4: Classification

Exercises 5, 10, 14 and 16

**An Introduction to Statistical Learning
with Applications in Python**

Name: Eliane UMWALI

Student ID: 101206

Chapter: 4 – Classification

Software: Python / Jupyter Notebook

Book: An Introduction to Statistical Learning with Applications in Python

August 24, 2026

Contents

1	Introduction	2
2	Exercise 5: LDA and QDA	2
2.1	(a) Linear Bayes Decision Boundary	2
2.2	(b) Non-linear Bayes Decision Boundary	2
2.3	(c) Effect of Increasing Sample Size	3
2.4	(d) True or False	3
3	Exercise 10: LDA When $p = 1$	3
4	Exercise 14: Auto Dataset	4
4.1	(a) Creating mpg01	4
4.2	(b) Graphical Exploration and Predictor Selection	5
4.3	(c) Training and Test Sets	6
4.4	(d) Linear Discriminant Analysis	6
4.5	(e) Quadratic Discriminant Analysis	7
4.6	(f) Logistic Regression	8
4.7	(g) Naive Bayes	8
4.8	(h) K-Nearest Neighbors	9
4.9	Exercise 14 Summary	10
5	Exercise 16: Boston Dataset	10
5.1	Creating the Binary Response	10
5.2	Predictor Exploration	11
5.3	Training and Test Sets	11
5.4	Logistic Regression	12
5.5	Linear Discriminant Analysis	12
5.6	Naive Bayes	12
5.7	K-Nearest Neighbors	12
5.8	Exercise 16 Summary	13
6	Overall Discussion	13
7	Conclusion	14
8	References	14

1 Introduction

Classification is a supervised statistical learning problem in which the response variable is qualitative rather than quantitative. The objective is to use one or more predictors to assign an observation to one of two or more classes.

Chapter 4 of *An Introduction to Statistical Learning with Applications in Python* introduces several important classification methods, including logistic regression, linear discriminant analysis (LDA), quadratic discriminant analysis (QDA), naive Bayes, and K-nearest neighbors (KNN).

This report addresses Exercises 5, 10, 14, and 16 from the chapter. The first two exercises focus on the theoretical differences between LDA and QDA and the derivation of the LDA discriminant function for one predictor. Exercises 14 and 16 apply classification methods to the Auto and Boston datasets.

For the computational exercises, the analysis was performed in Python using the ISLP datasets and scikit-learn classification methods.

2 Exercise 5: LDA and QDA

2.1 (a) Linear Bayes Decision Boundary

If the Bayes decision boundary is linear, QDA is expected to perform better on the training set because QDA is more flexible than LDA. Its greater flexibility allows it to fit the training observations at least as well as the more restricted LDA model.

However, on the test set, LDA is expected to perform better. When the true Bayes decision boundary is linear, the additional flexibility of QDA is not necessary. QDA estimates a separate covariance matrix for each class, which results in more parameters and higher variance.

Therefore:

- **Training set:** QDA is expected to perform better.
- **Test set:** LDA is expected to perform better.

The reason is the bias–variance trade-off. LDA is less flexible and therefore has lower variance, while QDA has greater flexibility but higher variance.

2.2 (b) Non-linear Bayes Decision Boundary

If the Bayes decision boundary is non-linear, QDA is expected to perform better on both the training and test sets.

LDA assumes a common covariance matrix across the classes and consequently produces a linear decision boundary. QDA allows each class to have its own covariance matrix and therefore produces a quadratic decision boundary.

Thus, when the true boundary is non-linear, QDA can represent the underlying relationship more accurately.

- **Training set:** QDA is expected to perform better.
- **Test set:** QDA is expected to perform better, provided there are enough observations to estimate the additional parameters reliably.

2.3 (c) Effect of Increasing Sample Size

As the sample size n increases, we expect the test prediction accuracy of QDA relative to LDA to improve.

QDA estimates a separate covariance matrix for each class and consequently estimates more parameters than LDA. With a small sample size, these estimates can have high variance. As the sample size increases, the parameter estimates become more stable and the variance of QDA decreases.

Therefore, QDA becomes more attractive relative to LDA as the sample size increases.

2.4 (d) True or False

False.

Although QDA is flexible enough to model a linear decision boundary, this additional flexibility does not guarantee a lower test error.

When the true Bayes decision boundary is linear, LDA already provides the appropriate form of decision boundary. QDA estimates more parameters and therefore has higher variance. The additional flexibility can lead to overfitting without providing a corresponding reduction in bias.

Therefore, when the true boundary is linear, LDA will generally have better test performance than QDA.

3 Exercise 10: LDA When $p = 1$

For LDA, the log posterior odds between class k and the reference class K can be written as

$$\log \left(\frac{\Pr(Y = k \mid X = x)}{\Pr(Y = K \mid X = x)} \right) = a_k + \sum_{j=1}^p b_{kj} x_j.$$

When $p = 1$, the mean vectors become scalars μ_k and μ_K , while the shared covariance matrix becomes the scalar variance σ^2 .

The expression becomes

$$\log \left(\frac{\Pr(Y = k \mid X = x)}{\Pr(Y = K \mid X = x)} \right) = \log \left(\frac{\pi_k}{\pi_K} \right) - \frac{(x - \mu_k)^2}{2\sigma^2} + \frac{(x - \mu_K)^2}{2\sigma^2}.$$

Expanding the squared terms gives

$$\log \left(\frac{\pi_k}{\pi_K} \right) - \frac{x^2 - 2x\mu_k + \mu_k^2}{2\sigma^2} + \frac{x^2 - 2x\mu_K + \mu_K^2}{2\sigma^2}.$$

The x^2 terms cancel, giving

$$\log \left(\frac{\pi_k}{\pi_K} \right) - \frac{\mu_k^2 - \mu_K^2}{2\sigma^2} + \frac{\mu_k - \mu_K}{\sigma^2} x.$$

Comparing this with

$$a_k + b_{k1}x,$$

we obtain

$$a_k = \log \left(\frac{\pi_k}{\pi_K} \right) - \frac{\mu_k^2 - \mu_K^2}{2\sigma^2}$$

and

$$b_{k1} = \frac{\mu_k - \mu_K}{\sigma^2}.$$

Thus, when there is only one predictor, the LDA log posterior odds are a linear function of x .

4 Exercise 14: Auto Dataset

The objective of this exercise is to predict whether a vehicle has high or low gas mileage using the Auto dataset.

4.1 (a) Creating mpg01

The first step is to create a binary response variable called `mpg01`. The median of MPG is used as the threshold.

Vehicles with MPG above the median are assigned a value of 1, while vehicles with MPG below the median are assigned a value of 0.

The Python implementation was:

```
1 from ISLP import load_data
2
3 Auto = load_data('Auto')
4
5 median_mpg = Auto['mpg'].median()
6
7 Auto['mpg01'] = (
8     Auto['mpg'] > median_mpg
9 ).astype(int)
10
11 print("Median MPG:", median_mpg)
```

The median MPG obtained from the dataset was

22.75.

Therefore,

$$\text{mpg01} = 1$$

represents vehicles with MPG above 22.75, while

$$\text{mpg01} = 0$$

represents vehicles with MPG below or equal to the median.

4.2 (b) Graphical Exploration and Predictor Selection

The relationship between `mpg01` and the available predictors was examined using graphical and numerical summaries.

The strongest associations were observed for:

- cylinders,
- weight,
- displacement,
- horsepower,
- year, and
- origin.

The strongest negative associations were observed for cylinders, weight, displacement, and horsepower. This indicates that vehicles with larger engines, more cylinders, greater horsepower, and greater weight tend to have lower fuel efficiency.

Year showed a positive association with `mpg01`, suggesting that newer vehicles tend to have better fuel efficiency.

Based on these findings, the following predictors were used in the classification models:

cylinders, displacement, horsepower, weight, year

4.3 (c) Training and Test Sets

The data were divided into training and test sets using a 50–50 split. The random state was set to 1 so that the results could be reproduced.

```
1 from sklearn.model_selection import train_test_split
2
3 features = [
4     'cylinders',
5     'displacement',
6     'horsepower',
7     'weight',
8     'year'
9 ]
10
11 X = Auto[features]
12 y = Auto['mpg01']
13
14 X_train, X_test, y_train, y_test = train_test_split(
15     X,
16     y,
17     test_size=0.5,
18     random_state=1,
19     stratify=y
20 )
```

The resulting datasets contained 196 observations each.

4.4 (d) Linear Discriminant Analysis

LDA was fitted using the selected predictors.

```
1 from sklearn.discriminant_analysis import
   LinearDiscriminantAnalysis
```

```
2 from sklearn.metrics import accuracy_score
3
4 lda = LinearDiscriminantAnalysis()
5
6 lda.fit(X_train, y_train)
7
8 lda_pred = lda.predict(X_test)
9
10 lda_error = 1 - accuracy_score(y_test, lda_pred)
11
12 print("LDA test error:", lda_error)
```

The test error obtained was approximately

9.69%.

Therefore, the corresponding test accuracy was approximately

90.31%.

4.5 (e) Quadratic Discriminant Analysis

QDA was fitted using the same predictors.

```
1 from sklearn.discriminant_analysis import
   QuadraticDiscriminantAnalysis
2
3 qda = QuadraticDiscriminantAnalysis()
4
5 qda.fit(X_train, y_train)
6
7 qda_pred = qda.predict(X_test)
8
9 qda_error = 1 - accuracy_score(y_test, qda_pred)
10
11 print("QDA test error:", qda_error)
```

The test error was approximately

11.73%.

The corresponding test accuracy was approximately

88.27%.

QDA performed slightly worse than LDA on this particular split, suggesting that the additional flexibility of QDA was not beneficial for this dataset.

4.6 (f) Logistic Regression

Logistic regression was fitted as follows:

```
1 from sklearn.linear_model import LogisticRegression
2
3 logit = LogisticRegression(max_iter=5000)
4
5 logit.fit(X_train, y_train)
6
7 logit_pred = logit.predict(X_test)
8
9 logit_error = 1 - accuracy_score(y_test, logit_pred)
10
11 print("Logistic regression test error:", logit_error)
```

The test error was approximately

10.20%,

giving a test accuracy of approximately

89.80%.

4.7 (g) Naive Bayes

A Gaussian naive Bayes classifier was fitted using the selected predictors.

```
1 from sklearn.naive_bayes import GaussianNB
2
3 nb = GaussianNB()
4
5 nb.fit(X_train, y_train)
6
7 nb_pred = nb.predict(X_test)
8
9 nb_error = 1 - accuracy_score(y_test, nb_pred)
10
```

```
11 print("Naive Bayes test error:", nb_error)
```

The test error was approximately

11.22%.

The corresponding accuracy was approximately

88.78%.

4.8 (h) K-Nearest Neighbors

KNN was evaluated using several values of K . Because KNN is based on distances, the predictors were standardized before fitting the classifier.

```
1 from sklearn.neighbors import KNeighborsClassifier
2 from sklearn.pipeline import make_pipeline
3 from sklearn.preprocessing import StandardScaler
4
5 for K in [1, 3, 5, 7, 9, 11]:
6
7     knn = make_pipeline(
8         StandardScaler(),
9         KNeighborsClassifier(n_neighbors=K)
10    )
11
12    knn.fit(X_train, y_train)
13
14    knn_pred = knn.predict(X_test)
15
16    error = 1 - accuracy_score(y_test, knn_pred)
17
18    print("K =", K, "Test error =", error)
```

The observed test errors were:

K	Test Error
1	7.65%
3	8.16%
5	9.18%
7	10.20%
9	11.22%
11	11.22%

Table 1: KNN test errors for the Auto dataset.

The best performance was obtained using

$$K = 1$$

with a test error of approximately

$$7.65\%.$$

4.9 Exercise 14 Summary

Method	Test Error	Test Accuracy
KNN ($K = 1$)	7.65%	92.35%
LDA	9.69%	90.31%
Logistic Regression	10.20%	89.80%
Naive Bayes	11.22%	88.78%
QDA	11.73%	88.27%

Table 2: Comparison of classification methods for the Auto dataset.

Among the methods tested, KNN with $K = 1$ produced the lowest test error. LDA was the second-best method, while QDA produced the highest test error among the five approaches.

The results also illustrate an important point from classification theory: greater model flexibility does not automatically result in better test performance.

5 Exercise 16: Boston Dataset

The objective of this exercise is to predict whether a Boston suburb has a crime rate above or below the median using classification methods.

5.1 Creating the Binary Response

The first step is to create a binary response variable called `crim01`.

```

1 from ISLP import load_data
2
3 Boston = load_data('Boston')
4
5 median_crim = Boston['crim'].median()
6
7 Boston['crim01'] = (
8     Boston['crim'] > median_crim

```

```
9 ).astype(int)
```

The response variable is interpreted as follows:

- `crim01 = 0`: crime rate below or equal to the median.
- `crim01 = 1`: crime rate above the median.

The median crime rate was approximately

0.25651.

5.2 Predictor Exploration

The available predictors were examined to identify variables that were most strongly associated with `crim01`.

The variables selected for the classification models were:

`nox, rad, age, tax, indus, dis`

These variables showed relatively strong associations with whether a suburb had a crime rate above or below the median.

5.3 Training and Test Sets

The observations were divided into training and test sets using a 50–50 split, with `random_state=1` and stratification.

```
1 from sklearn.model_selection import train_test_split
2
3 features = [
4     'nox',
5     'rad',
6     'age',
7     'tax',
8     'indus',
9     'dis'
10 ]
11
12 X = Boston[features]
13 y = Boston['crim01']
14
15 X_train, X_test, y_train, y_test = train_test_split(
16     X,
```

```
17     y ,  
18     test_size=0.5 ,  
19     random_state=1 ,  
20     stratify=y  
21 )
```

5.4 Logistic Regression

The logistic regression model produced a test error of approximately

15.42%.

The corresponding test accuracy was approximately

84.58%.

5.5 Linear Discriminant Analysis

LDA produced a test error of approximately

16.21%.

The corresponding accuracy was approximately

83.79%.

5.6 Naive Bayes

Gaussian naive Bayes produced a test error of approximately

17.00%.

The corresponding accuracy was approximately

83.00%.

5.7 K-Nearest Neighbors

Several values of K were evaluated.

K	Test Error
1	6.72%
3	7.91%
5	7.51%
10	8.30%

Table 3: KNN test errors for the Boston dataset.

The best result was obtained using

$$K = 1$$

with a test error of approximately

$$6.72\%.$$

5.8 Exercise 16 Summary

Method	Test Error	Test Accuracy
KNN ($K = 1$)	6.72%	93.28%
Logistic Regression	15.42%	84.58%
LDA	16.21%	83.79%
Naive Bayes	17.00%	83.00%

Table 4: Comparison of classification methods for the Boston dataset.

KNN with $K = 1$ achieved the lowest test error among the methods considered. Its performance was substantially better than logistic regression, LDA, and naive Bayes for this particular train-test split.

6 Overall Discussion

The four exercises demonstrate several important principles of classification.

First, LDA and QDA differ mainly in their assumptions about the covariance structure of the predictors. LDA assumes a common covariance matrix across classes and therefore produces a linear decision boundary. QDA allows class-specific covariance matrices and can therefore produce a quadratic decision boundary.

Second, the additional flexibility of a model does not necessarily result in better test performance. QDA can fit more complex relationships than LDA, but this flexibility can increase variance and lead to poorer predictions when the additional complexity is unnecessary.

Third, the computational exercises demonstrate that classification performance depends strongly on the dataset. There is no single classification method that is guaranteed to outperform all others in every problem.

For the Auto dataset, KNN with $K = 1$ achieved the lowest test error among the methods evaluated. For the Boston dataset, KNN with $K = 1$ also achieved the lowest test error among the tested models.

The KNN results should nevertheless be interpreted carefully. KNN is a non-parametric method and can have high variance, particularly when a small value of K is used. The ISLP text notes that the choice of K has an important effect on the flexibility of KNN and that $K = 1$ can produce a very flexible classifier.

7 Conclusion

This chapter provided practical experience with several fundamental classification methods: logistic regression, LDA, QDA, naive Bayes, and KNN.

Exercise 5 demonstrated the bias-variance trade-off between LDA and QDA. LDA is generally preferable when the true decision boundary is linear and the sample size is relatively small, while QDA becomes more attractive when the decision boundary is non-linear or when a sufficiently large sample is available.

Exercise 10 showed mathematically that when there is one predictor, the LDA log posterior odds are a linear function of that predictor.

Exercises 14 and 16 demonstrated the application of classification methods to real datasets. In the particular train-test splits used in this analysis, KNN with $K = 1$ produced the lowest test error for both the Auto and Boston datasets.

The exact test error rates depend on the training-test split. Therefore, the reported values should be interpreted as the results for the specified 50-50 split with `random_state=1`, rather than as universal properties of the datasets.

Overall, the exercises demonstrate why model selection should be based on test performance and an understanding of the assumptions and flexibility of each classification method rather than simply choosing the most complex model.

8 References

1. James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. *An Introduction to Statistical Learning with Applications in Python*. Springer, 2023.
2. ISLP Python package and datasets used for the computational exercises.
3. Scikit-learn documentation for classification methods including Logistic Regression,

Linear Discriminant Analysis, Quadratic Discriminant Analysis, Gaussian Naive Bayes, and K-Nearest Neighbors.